

Full-Stack .NET Developer

Interview Preparation Notes

Topics Covered: C# & .NET | SOLID & Design Patterns | ASP.NET MVC | EF Core | React & Next.js | NextAuth | System Design | PostgreSQL & SQL | DSA

■ *Tailored for 3+ years experience — covers depth, not just basics.*

1 · C# & .NET Core — Fundamentals

1.1 OOP Pillars

Pillar	Definition	Real Example
Encapsulation	Bundle data + methods; hide internals via access modifiers	Private fields + public properties in a model class
Inheritance	Derive class from base to reuse/extend behaviour	EmployeeController : BaseController
Polymorphism	One interface, many implementations (overriding/overloading)	Animal.Speak() → Dog.Speak(), Cat.Speak()
Abstraction	Expose only essentials; hide complexity	IRepository hides DB details from service layer

1.2 SOLID Principles

S — Single Responsibility

Rule: One class, one reason to change.

Example: InvoiceService only calculates invoices; InvoicePrinter only prints.

O — Open/Closed

Rule: Open for extension, closed for modification.

Example: Add new discount type via new class, not editing existing if/else chain.

L — Liskov Substitution

Rule: Subtypes must be substitutable for base types.

Example: Square should NOT extend Rectangle if it breaks area semantics.

I — Interface Segregation

Rule: Many small interfaces > one fat interface.

Example: ISave, IDelete instead of one ICRUD forcing unused methods.

D — Dependency Inversion

Rule: Depend on abstractions, not concretions.

Example: Inject IEmailService; swap SMTP↔SendGrid without changing business logic.

1.3 async / await

async/await is syntactic sugar over Tasks. The compiler rewrites the method as a state machine.

- Always use **await** — never **.Result** or **.Wait()** (causes deadlocks in ASP.NET).
- Return **Task** not **void** for async methods (except event handlers).
- **ConfigureAwait(false)** in library code to avoid context capture overhead.
- Use **Task.WhenAll()** for concurrent independent calls.

```
// Correct pattern
public async Task GetDataAsync()
```

```

{
var result = await _httpClient.GetStringAsync(url);
return result;
}

// Concurrent calls
var (a, b) = await (Task1Async(), Task2Async()); // WRONG - no tuple await
var t1 = Task1Async(); var t2 = Task2Async();
await Task.WhenAll(t1, t2);

```

1.4 EF Core Essentials

- **DbContext** is a Unit of Work + Repository. Register as **Scoped**.
- **Migrations**: *dotnet ef migrations add* → *dotnet ef database update*.
- Use **AsNoTracking()** for read-only queries (big perf gain).
- **Eager loading**: `.Include(e => e.Orders)`. **Lazy loading**: enable via proxies.
- Avoid N+1: always Include related data or use projections (`.Select()`).

```

// Good – projection avoids loading full entity
var names = await _db.Employees
.Where(e => e.IsActive)
.AsNoTracking()
.Select(e => new { e.Id, e.Name })
.ToListAsync();

```

1.5 Design Patterns — Quick Reference

Pattern	Type	Use Case	Key Participants
Singleton	Creational	Shared config/logger instance	Private ctor, static Instance
Repository	Structural	Abstract DB access from business logic	IRepository, concrete impl
Factory Method	Creational	Create objects without specifying class	Creator, ConcreteCreator
Strategy	Behavioural	Swap algorithm at runtime	IStrategy, ConcreteStrategy, Context
Observer	Behavioural	Event-driven notifications	IObserver, IObservable / C# events
Decorator	Structural	Add behaviour without subclassing	IComponent, Decorator wrapping it
CQRS	Architectural	Separate read & write models	Commands, Queries, Handlers
Mediator	Behavioural	Decouple components via mediator	MediatR: IRequest, IRequestHandler

2 · ASP.NET MVC & Web API

2.1 MVC Request Pipeline

Request → **Routing** → **Controller** → **Action** → **View/JSON**. Middlewares run before routing (in Program.cs order).

- **Filters:** Authorization → Resource → Action → Exception → Result.
- **Model Binding:** binds route values, query strings, body to action params.
- **[FromBody]** for JSON, **[FromRoute]** for route params, **[FromQuery]** for query strings.

2.2 REST API Best Practices

HTTP Verb	Usage	Status Codes
GET	Read resource	200 OK, 404 Not Found
POST	Create resource	201 Created, 400 Bad Request
PUT	Full update	200 OK, 204 No Content
PATCH	Partial update	200 OK, 204 No Content
DELETE	Remove resource	204 No Content, 404 Not Found

- Always version APIs: /api/v1/assets
- Return Problem Details (RFC 7807) for errors.
- Use ActionResult<T> for strong return types.

2.3 Dependency Injection Lifetimes

Lifetime	Created	Destroyed	Use For
Singleton	App start	App stop	Config, Logger, Cache
Scoped	Per request	End of request	DbContext, per-request services
Transient	Every resolve	After use	Lightweight, stateless services

3 · React & Next.js 14

3.1 Core React Hooks

useState

Local state. Re-renders on change.

```
const [count, setCount] = useState(0);
```

useEffect

Side effects (fetch, subscriptions, timers). Runs after render.

```
useEffect(() => { fetchData(); }, [dep]); // dep array controls re-run
```

useCallback

Memoize function reference — prevents child re-renders.

```
const fn = useCallback(() => doWork(id), [id]);
```

useMemo

Memoize expensive computation.

```
const sorted = useMemo(() => list.sort(), [list]);
```

useRef

Persist value without re-render; direct DOM access.

```
const timerRef = useRef(null);
```

useContext

Consume React context (e.g. theme, session).

```
const session = useContext(SessionContext);
```

useReducer

Complex state logic — like local Redux.

```
const [state, dispatch] = useReducer(reducer, init);
```

3.2 Next.js 14 App Router vs Pages Router

Feature	Pages Router	App Router (Next 13+)
File convention	pages/xyz.tsx	app/xyz/page.tsx
Data fetching	getServerSideProps / getStaticProps	async Server Components / fetch()
Layouts	_app.tsx	layout.tsx (nested)
Loading states	Manual	loading.tsx (auto Suspense)
Error boundaries	Manual	error.tsx
API Routes	pages/api/	app/api/route.ts

■ *Next.js 14.x + NextAuth v4: use Pages Router (app router requires NextAuth v5 beta).*

3.3 NextAuth v4 — JWT Flow

- Configure in **pages/api/auth/[...nextauth].ts**.
- JWT callback: enrich token with extra fields (userId, role, accessToken).
- Session callback: expose token fields to client via **useSession()**.
- Use **getServerSession()** on server side (not getSession).

```
// pages/api/auth/[...nextauth].ts
jwt: async ({ token, user }) => {
  if (user) {
    token.userId = user.id;
    token.role = user.role;
  }
  return token;
},
session: async ({ session, token }) => {
  session.user.userId = token.userId as string;
  session.user.role = token.role as string;
  return session;
},
```

3.4 Session Idle Timeout Pattern

Use **localStorage deadline timestamp** (not setInterval countdown) to survive tab pause/sleep.

```
// On activity → reset deadline
const resetTimer = () => {
  const deadline = Date.now() + IDLE_TIMEOUT_MS;
  localStorage.setItem('idleDeadline', String(deadline));
};

// Every second check remaining
setInterval(() => {
  const deadline = Number(localStorage.getItem('idleDeadline') || 0);
  const remaining = Math.max(0, deadline - Date.now());
  if (remaining === 0) signOut();
}, 1000);

// Handle tab visibility restore
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'visible') checkDeadlineNow();
});
```

4 · System Design — Key Concepts

4.1 Approach Framework (SD Interview)

1. Clarify Requirements

Functional (what it does) vs Non-functional (scale, latency, availability).

2. Estimate Scale

DAU, QPS, storage needed. Back-of-envelope math.

3. High-Level Design

Draw major components: Client → LB → API Servers → DB/Cache.

4. Deep Dive

Focus on bottlenecks: DB schema, caching, message queues, CDN.

5. Identify Bottlenecks

Single points of failure, hotspots, scaling strategies.

4.2 Scalability Patterns

Pattern	When to Use	Trade-off
Horizontal Scaling	Stateless services need more capacity	Need sticky sessions or shared state
Caching (Redis)	Reduce DB load for frequent reads	Cache invalidation complexity
CDN	Static assets, global users	Cost; stale content risk
Database Sharding	DB is the bottleneck	Complex queries, cross-shard joins hard
Read Replicas	Read-heavy workloads	Eventual consistency on replica
Message Queue (RabbitMQ/Kafka)	Async processing, decoupling services	Added infra complexity
Rate Limiting	Protect APIs from abuse	May block legit traffic if misconfigured

4.3 CAP Theorem

A distributed system can guarantee only **2 of 3**: **C**onsistency, **A**vailability, **P**artition tolerance.

- **CP systems** (e.g. HBase, Zookeeper): consistent but may be unavailable during partition.
- **AP systems** (e.g. Cassandra, DynamoDB): available but may return stale data.
- **CA**: only possible without network partitions (impractical in distributed systems).

4.4 Common Interview System Designs

System	Key Components	Main Challenges
URL Shortener	Hash function, KV store (Redis), DB	Hash collision, 301 vs 302 redirect

System	Key Components	Main Challenges
Chat App	WebSocket, message queue, DB	Message ordering, offline delivery
File Upload (like yours)	Object storage (S3), metadata DB, CDN	Large file chunking, duplicate detection
Auth System	JWT/OAuth, token store, refresh flow	Token revocation, session expiry
Rate Limiter	Token bucket / sliding window, Redis	Distributed counter sync

5 · PostgreSQL & SQL

5.1 Joins — Visual Reference

Join Type	Returns	Use When
INNER JOIN	Rows matching in both tables	Need related data in both
LEFT JOIN	All left + matching right (NULLs for no match)	Want all left rows regardless
RIGHT JOIN	All right + matching left	Want all right rows regardless
FULL OUTER JOIN	All rows from both (NULLs where no match)	Find unmatched rows on either side
CROSS JOIN	Cartesian product	Generate combinations
SELF JOIN	Table joined with itself	Org hierarchy, find pairs

5.2 Window Functions

Window functions operate across a set of rows related to the current row WITHOUT collapsing them.

```
SELECT
  employee_id,
  department,
  salary,
  RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank,
  ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num,
  LAG(salary) OVER (PARTITION BY department ORDER BY salary) AS prev_salary,
  SUM(salary) OVER (PARTITION BY department) AS dept_total
FROM employees;
```

Function	Description
ROW_NUMBER()	Unique sequential number per partition
RANK()	Rank with gaps on ties (1,1,3)
DENSE_RANK()	Rank without gaps on ties (1,1,2)
LAG(col, n)	Value from n rows before
LEAD(col, n)	Value from n rows ahead
NTILE(n)	Divide rows into n buckets

5.3 Indexes & Performance

- **B-tree index** (default): good for equality and range queries.
- **Hash index**: only equality checks; faster for = but can't do ranges.
- **Partial index**: index a subset of rows. Very efficient.
- **Composite index**: order matters — (a,b) index helps queries on a, or a+b, NOT just b.
- **EXPLAIN ANALYZE**: always check query plan before tuning.

```
-- Partial index example (only active records)
CREATE INDEX idx_active_employees ON employees(name) WHERE is_active = true;
```

```
-- Check query plan
EXPLAIN ANALYZE SELECT * FROM employees WHERE is_active = true AND name = 'Swaroop';
```

5.4 Transactions & Isolation Levels

Level	Dirty Read	Non-repeatable Read	Phantom Read
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED (PG default)	No	Yes	Yes
REPEATABLE READ	No	No	Yes
SERIALIZABLE	No	No	No

- **Deadlock**: two txns wait for each other's lock. PG auto-detects & rolls back one.
- Always use **BEGIN/COMMIT/ROLLBACK** explicitly for multi-step ops.

5.5 PostgreSQL Specifics

- **SERIAL / BIGSERIAL**: auto-increment. Fix desync with **setval(seq, max_id)**.
- **JSONB**: binary JSON — indexable. JSON: text storage. Prefer JSONB.
- **CTEs (WITH clause)**: named subqueries, improve readability.
- **VACUUM / ANALYZE**: reclaim dead tuples; update planner statistics.
- **pg_stat_activity**: view running queries; detect long locks.

```
-- Fix sequence desync
SELECT setval('gen.table_id_seq', (SELECT MAX(id) FROM gen.table_name));

-- Find blocking queries
SELECT pid, query, state, wait_event_type
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;
```

6 · Data Structures & Algorithms

6.1 Big-O Cheat Sheet

Operation	Array	Linked List	Hash Map	BST (balanced)
Access	$O(1)$	$O(n)$	$O(1)$ avg	$O(\log n)$
Search	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$
Insert (end)	$O(1)$ amort	$O(1)$	$O(1)$ avg	$O(\log n)$
Insert (mid)	$O(n)$	$O(1)$	—	$O(\log n)$
Delete	$O(n)$	$O(1)$	$O(1)$ avg	$O(\log n)$

6.2 Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

6.3 Common Patterns

Two Pointers

When: Array/string problems. One from each end or both moving forward.

Examples: Two Sum (sorted), palindrome check, remove duplicates.

Sliding Window

When: Contiguous subarray/substring problems.

Examples: Max sum subarray of size k, longest substring without repeat.

Fast & Slow Pointers

When: Cycle detection in linked lists.

Examples: Floyd's cycle detection, find middle of linked list.

Binary Search

When: Sorted array. $O(\log n)$ search.

Examples: Find target, first/last occurrence, search in rotated array.

BFS / Level Order

When: Shortest path, level-by-level tree traversal.

Examples: Shortest path in unweighted graph, word ladder.

DFS / Backtracking

When: Explore all paths, generate permutations/subsets.

Examples: N-Queens, combination sum, maze traversal.

Dynamic Programming

When: Overlapping subproblems + optimal substructure.

Examples: Fibonacci, 0/1 Knapsack, LCS, coin change.

7 · Security & Best Practices

7.1 Web Security Essentials

Threat	Description	Mitigation
SQL Injection	Malicious SQL via user input	Parameterized queries / EF Core (ORM)
XSS	Inject scripts into page	Encode output; CSP headers; HttpOnly cookies
CSRF	Forged requests from other sites	Anti-forgery tokens; SameSite cookies
IDOR	Access other users' resources by guessing IDs	Always check ownership server-side
Broken Auth	Weak tokens, no expiry	Short-lived JWT; secure refresh token rotation
Sensitive Data Exposure	Secrets in code/logs	Use .env; never log secrets; encrypt at rest

7.2 JWT Best Practices

- Keep **access tokens short-lived** (15 min). Use **refresh tokens** (7–30 days) for renewal.
- Store refresh token in **HttpOnly cookie**; access token in memory (not localStorage).
- Always validate **exp, iss, aud** claims on server.
- Use **RS256** (asymmetric) for distributed systems; **HS256** ok for single-server.
- Implement **token rotation**: on refresh, invalidate old refresh token.

7.3 Secrets Management

- NEVER hardcode secrets in source code or **next.config.js**.
- Use **.env.local** for local dev; CI/CD secrets for staging/prod.
- If accidentally committed: **rotate the key immediately**, then remove from history.
- Use **NEXTAUTH_SECRET** generated via: `openssl rand -base64 32`

```
# .env.local
NEXTAUTH_SECRET=your-generated-secret
NEXTAUTH_URL=http://localhost:3000
API_BASE_URL=https://api.yourdomain.com
DB_CONNECTION_STRING=Host=...;Username=...;Password=...
```

8 · Quick-Fire Interview Q&A

C# / .NET

Q: What is the difference between interface and abstract class?

A: Interface: pure contract, multiple inheritance, no implementation (C#8+ can have defaults). Abstract class: partial implementation, single inheritance, can have state.

Q: What is the difference between IEnumerable and IQueryable?

A: IEnumerable executes in memory (LINQ to Objects). IQueryable translates to SQL (EF Core). Use IQueryable in repositories; return IEnumerable to service layer after materializing.

Q: Explain value types vs reference types.

A: Value types (int, struct, enum) stored on stack; copied on assignment. Reference types (class, string, array) stored on heap; assignment copies the reference.

Q: What is boxing/unboxing?

A: Boxing: wrapping a value type in object (heap allocation). Unboxing: casting back. Avoid in hot paths — causes GC pressure.

Q: What is a deadlock in async code?

A: Calling .Result or .Wait() on a Task in a sync context that has only one thread (e.g. ASP.NET Framework). The async continuation needs the thread that's blocked. Fix: use await throughout.

React / Next.js

Q: What is the difference between useMemo and useCallback?

A: useMemo memoizes a computed value. useCallback memoizes a function reference. Both prevent unnecessary re-computation/re-renders when dependencies haven't changed.

Q: What is hydration in Next.js?

A: Server renders HTML; browser downloads JS and 'hydrates' — attaches React event handlers to existing DOM. Mismatch between server and client HTML causes hydration errors.

Q: When would you use getServerSideProps vs getStaticProps?

A: getStaticProps: data doesn't change per-request (blogs, docs). Build-time fetch. getServerSideProps: data changes per-request (user-specific, real-time). Request-time fetch.

Q: What is React reconciliation?

A: React's diffing algorithm to determine minimal DOM updates. Uses virtual DOM. key prop helps React identify which items changed in lists.

System Design

Q: How would you design a file upload system?

A: Client → pre-signed URL from API → upload directly to S3/object store. Store metadata (filename, size, uploader, timestamp) in DB. Use background job for virus scan, thumbnail generation. CDN for delivery.

Q: How do you handle database migrations in production?

A: Use backward-compatible changes (add column before dropping old one). Run migrations before deploying new code (Blue-Green). Test in staging first. Always backup before running migrations.

Good luck with your interviews, Swaroop! ■

Remember: explain your thinking out loud, relate answers to real projects you've worked on, and ask clarifying questions before diving into system design.